

Relatório de Teste de Usabilidade do GitHub

1. Introdução

1.1. Contexto do projeto:

O sistema escolhido foi o **GitHub Web**, plataforma utilizada para hospedagem de código, controle de versões e colaboração em projetos de software. Seu público-alvo é composto principalmente por desenvolvedores, estudantes e equipes de tecnologia. Os testes serão realizados no repositório API---Test, avaliando aspectos de navegação, fluxos, mensagens e informações apresentadas pelo sistema.

1.2. Objetivos do teste:

Avaliar a usabilidade do GitHub Web para seu público-alvo, verificando se as principais tarefas podem ser realizadas de forma clara e eficiente, com base nas 10 heurísticas de Nielsen.

2. Metodologia

2.1. Planejamento do teste:

Heurística	Interface	Cenário (ação)	Testador
6. Reconhecimento em vez de memorização	Dashboard	Na página inicial do GitHub, localizar e abrir o repositório API---Test na lista de repositórios.	Desenvolvedor
4. Consistência e padrões	Navegação principal do repositório	No API---Test, abrir a aba Issues, depois a aba Pull requests e retornar à aba Code usando a mesma barra de navegação do repositório.	Desenvolvedor
8. Design estético e minimalista	Página principal do repositório	Na página, principal do API---Test, sem rolar a página, identificar qual é a branch padrão exibida, distinguindo-a dos demais elementos do cabeçalho.	Desenvolvedor
4. Consistência e padrões	Navegação de arquivos	Abrir a pasta src e retornar à raiz do repositório usando o caminho de navegação exibido na página.	Desenvolvedor
7. Flexibilidade e eficiência de uso	Menu Code	Copiar a URL HTTPS do repositório.	Desenvolvedor
2. Correspondência entre o sistema e o mundo real	Issue - classificação do problema	Na Issue de teste disponível no repositório, indicar que o problema encontrado é um bug.	Desenvolvedor

Heurística	Interface	Cenário (ação)	Testador
2. Correspondência entre o sistema e o mundo real	Ações do repositório	Antes de acessar o histórico, perguntar ao participante o que ele espera que aconteça se ele clicar em um commit específico (ex.: vai ver o quê?); depois abrir o commit e comparar a expectativa com o que é exibido.	Desenvolvedor
8. Design estético e minimalista	Histórico de alterações	No histórico de alterações, identificar o autor do commit mais recente, a mensagem e a data, sem abrir nenhum item.	Desenvolvedor
6. Reconhecimento em vez de memorização	Seletor de branches	Abrir o seletor de branches e selecionar a branch de testes.	Desenvolvedor
1. Visibilidade do status do sistema	Criação de branch	A partir da main, criar a branch teste-usabilidade e permanecer nela após a criação.	Desenvolvedor
5. Prevenção de erros	Criação de branch	Tentar criar uma branch com o nome refs/teste e observar a resposta do GitHub.	Desenvolvedor
9. Reconhecimento, diagnóstico e recuperação de erros	Criação de branch	Após a tentativa inválida, utilizar a mensagem ou indicação apresentada pelo GitHub para corrigir o nome da branch e concluir a criação.	Desenvolvedor
7. Flexibilidade e eficiência de uso	Go to file	Usar Go to file para localizar e abrir o arquivo pom.xml.	Desenvolvedor
3. Controle e liberdade para o usuário	Edição de arquivo	Iniciar a edição do README.md, digitar “alteração temporária” e sair da edição sem salvar a mudança.	Desenvolvedor
1. Visibilidade do status do sistema	Edição e commit	Na branch teste-usabilidade, editar o README.md, adicionar “Teste de usabilidade do GitHub Web” e salvar a alteração com um commit.	Desenvolvedor
8. Design estético e minimalista	Página principal do repositório	Na página principal do API---Test, localizar quantas branches e tags estão disponíveis no repositório.	Desenvolvedor

Heurística	Interface	Cenário (ação)	Testador
2. Correspondência entre o sistema e o mundo real	Pull Requests	Antes de abrir a PR, pedir ao participante que explique o que acontecera ao enviar as alterações da branch teste-usabilidade para main; em seguida, enviar as alterações e comparar com o resultado real.	Desenvolvedor
8. Design estético e minimalista	Pull Request	Na PR criada, identificar o status de mergabilidade em meio às demais informações exibidas na mesma área.	Desenvolvedor
4. Consistência e padrões	Controles da Pull Request	Na Pull Request, abrir os seletores Assignees e Labels e fechar os dois sem realizar alterações.	Desenvolvedor
3. Controle e liberdade para o usuário	Edição da Pull Request	Iniciar a edição do título da Pull Request, alterar temporariamente o texto e cancelar a edição.	Desenvolvedor
5. Prevenção de erros	Merge da Pull Request	Iniciar o processo de merge da Pull Request e parar antes da confirmação final.	Desenvolvedor
9. Reconhecimento, diagnóstico e recuperação de erros	Comparação de branches	Abrir a comparação de branches, selecionar main como base e main como comparação e, após o retorno do GitHub indicando ausência de diferenças, alterar a comparação para teste-usabilidade até as diferenças aparecerem.	Desenvolvedor
2. Correspondência entre o sistema e o mundo real	Issues	Antes de iniciar, pedir ao participante que explique a diferença esperada entre issues e Pull Requests; em seguida abrir onde é possível registrar um novo problema e comparar com a expectativa inicial.	Desenvolvedor
5. Prevenção de erros	Criação de Issue	No formulário de nova Issue, deixar o título vazio e tentar concluir a criação.	Desenvolvedor
1. Visibilidade do status do sistema	Criação de Issue	Preencher o título “Teste de usabilidade - Issue”, adicionar uma descrição curta e criar a Issue.	Desenvolvedor
6. Reconhecimento em vez de memorização	Labels da Issue	Na Issue criada, definir a própria conta de teste como responsável, escolhendo-a entre as opções apresentadas pelo GitHub	Desenvolvedor
4. Consistência e padrões	Criação em Issues e Pull Requests	Na lista de Issues, localizar o controle para criar uma Issue; depois, na lista de Pull Requests, localizar o controle para criar uma Pull Request, sem concluir nenhuma criação.	Desenvolvedor

Heurística	Interface	Cenário (ação)	Testador
7. Flexibilidade e eficiência de uso	Lista de Issues	Na lista de Issues, usar os filtros para mostrar apenas Issues abertas com a label bug.	Desenvolvedor
9. Reconhecimento, diagnóstico e recuperação de erros	Filtros de Issues	Na lista de Issues, aplicar um filtro que não retorne resultados e, a partir da tela apresentada pelo GitHub, remover o filtro até recuperar a lista normal.	Desenvolvedor
3. Controle e liberdade para o usuário	Assignee da Issue	Na Issue criada, adicionar um comentário e, em seguida, excluí-lo, verificando se a exclusão é refletida imediatamente.	Desenvolvedor
1. Visibilidade do status do sistema	Estado da Issue	Fechar a Issue criada e permanecer na página até o estado Closed ser exibido.	Desenvolvedor
3. Controle e liberdade para o usuário	Estado da Issue	Reabrir a Issue fechada e permanecer na página até o estado Open ser exibido novamente.	Desenvolvedor
5. Prevenção de erros	Criação de repositório	Abrir New repository, informar API---Test como nome do novo repositório e observar a validação sem criar outro repositório.	Desenvolvedor
6. Reconhecimento em vez de memorização	Menu da conta	Abrir o menu do avatar da conta e acessar Settings pela opção exibida no menu.	Desenvolvedor
7. Flexibilidade e eficiência de uso	Lista de branches	Abrir a página de branches, filtrar pelos nomes que contêm "teste-" e abrir a branch teste-usabilidade.	Desenvolvedor
9. Reconhecimento, diagnóstico e recuperação de erros	Página não encontrada	Abrir https://github.com/ContaDeTesteTesteDeUsabilidade/RepositorioInexistente e retornar ao API---Test usando a navegação do GitHub.	Desenvolvedor
10. Ajuda e documentação	GitHub Docs	A partir de um recurso de ajuda do GitHub, abrir a documentação oficial.	Desenvolvedor

Heurística	Interface	Cenário (ação)	Testador
10. Ajuda e documentação	Atalhos do teclado	Abrir a janela de atalhos disponíveis e depois fechá-la.	Desenvolvedor
10. Ajuda e documentação	Busca na documentação	Na documentação oficial do GitHub, usar a busca para localizar uma orientação sobre como criar uma Issue e abrir um resultado correspondente.	Desenvolvedor
10. Ajuda e documentação	GitHub Support	A partir dos recursos de ajuda do GitHub, abrir a página de suporte do GitHub.	Desenvolvedor

2.2. Técnica de teste utilizada:

Será aplicado teste de caixa preta. Serão considerados apenas as ações do participante e os resultados visíveis apresentados pelo GitHub Web.

2.3. Ferramentas e recursos:

- Firefox.
- Conta ContaDeTesteTesteDeUsabilidade.
- Repositório <https://github.com/ContaDeTesteTesteDeUsabilidade/API---Test>.
- OBS Studio.
- Tabela de resultados.
- Playlist no YouTube para as evidências.

3. Execução do Teste

3.1. Processo de teste e resultados obtidos:

Equipe na sessão:

- Ronald aplica os testes;
- Bernardo Roche registra as informações;
- Gustavo observa;
- Christopher revisa os registros, evidências e relatório.

Para cada CT serão registrados: SD (sem dificuldade), CD (com dificuldade) ou NC (não concluiu),

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT01	SD	8,0s	O participante localizou e acessou o repositório solicitado sem apresentar dificuldades durante a execução.	Link: CT1

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT02	SD	13,0s	O participante navegou entre as abas Issues, Pull Requests e Code sem apresentar dificuldades, compreendendo o padrão de navegação utilizado pelo GitHub.	Link: CT2
CT03	SD	10,0s	O participante identificou a branch padrão exibida na página principal do repositório sem apresentar dificuldades.	Link: CT3
CT04	CD	22,0s	O participante apresentou pequena dificuldade para identificar o recurso de navegação necessário para retornar à página principal do repositório.	Link: CT4
CT05	SD	10,0s	O participante localizou e copiou a URL HTTPS do repositório sem apresentar dificuldades.	Link: CT5
CT06	CD	44,0s	O participante apresentou dificuldade para localizar o recurso utilizado para indicar que a Issue correspondia a um bug.	Link: CT6

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT07	SD	20,0s	Antes da execução, o participante informou que esperava visualizar os arquivos modificados ao acessar um commit. Após a abertura, o resultado apresentado pelo GitHub correspondeu à expectativa informada.	Link: CT7
CT08	SD	16,0s	O participante identificou o autor, a mensagem e a data do commit mais recente sem precisar abrir o item e sem apresentar dificuldades.	Link: CT8
CT09	SD	15,0s	O participante utilizou o seletor de branches e localizou a branch de testes sem apresentar dificuldades.	Link: CT9
CT10	SD	40,0s	O participante criou a branch teste-usabilidade e identificou corretamente que permaneceu nela após a criação, sem apresentar dificuldades.	Link: CT10
CT11	SD	22,0s	O participante realizou a tentativa sem dificuldade. O GitHub impediu a criação da branch com o nome informado, porém não apresentou de forma clara o motivo da restrição.	Link: CT11

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT12	CD	34,0s	O participante apresentou dificuldade para identificar, a partir da resposta do GitHub, o motivo pelo qual o nome informado para a branch era considerado inválido.	Link: CT12
CT13	SD	13,0s	O participante utilizou o recurso Go to file para localizar e abrir o arquivo pom.xml sem apresentar dificuldades.	Link: CT13
CT14	SD	35,0s	O participante iniciou a edição do arquivo e conseguiu sair sem salvar a alteração temporária, sem apresentar dificuldades.	Link: CT14
CT15	SD	57,0s	O participante realizou a edição do README e concluiu o commit da alteração sem apresentar dificuldades.	Link: CT15
CT16	SD	15,0s	O participante localizou as informações referentes às branches e tags disponíveis no repositório sem apresentar dificuldades.	Link: CT16
CT17	SD	32,0s	Antes da execução, o participante informou que esperava que as alterações realizadas fossem incorporadas à branch main. A tarefa foi executada sem dificuldades.	Link: CT17

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT18	CD	42,0s	O participante apresentou pequena dificuldade para localizar o status de possibilidade de merge da Pull Request devido à quantidade de informações exibidas na mesma área da interface.	Link: CT18
CT19	SD	17,0s	O participante localizou e utilizou os seletores Assignees e Labels da Pull Request sem apresentar dificuldades.	Link: CT19
CT20	NC	1m 15s	O participante não conseguiu concluir a tarefa, pois não localizou na interface o recurso necessário para editar o título da Pull Request.	Link: CT20
CT21	SD	22,0s	O participante iniciou o processo de merge e identificou a etapa de confirmação antes da conclusão da ação, sem apresentar dificuldades.	Link: CT21
CT22	SD	29,0s	O participante realizou a comparação entre as branches e conseguiu alterar a seleção até visualizar as diferenças esperadas, sem apresentar dificuldades.	Link: CT22

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT23	SD	28,0s	Antes da execução, o participante informou que compreendia Issues como registros de problemas e Pull Requests como recursos relacionados à atualização do projeto. A tarefa foi concluída sem dificuldades.	Link: CT23
CT24	SD	18,0s	O participante identificou sem dificuldades que o GitHub não permite a criação de uma Issue sem o preenchimento do título obrigatório.	Link: CT24
CT25	SD	34,0s	O participante preencheu as informações solicitadas e criou a Issue sem apresentar dificuldades.	Link: CT25
CT26	CD	59,0s	O participante apresentou dificuldade para localizar o recurso utilizado para atribuir a própria conta como responsável pela Issue.	Link: CT26
CT27	SD	13,0s	O participante localizou os controles utilizados para iniciar a criação de uma nova Issue e de uma nova Pull Request sem apresentar dificuldades.	Link: CT27
CT28	SD	37,0s	O participante utilizou os filtros para exibir apenas Issues abertas associadas à classificação bug, sem apresentar dificuldades.	Link: CT28

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT29	SD	19,0s	O participante aplicou um filtro sem resultados e identificou claramente que nenhuma Issue correspondia aos critérios utilizados. Em seguida, recuperou a listagem normalmente.	Link: CT29
CT30	SD	24,0s	O participante adicionou e excluiu o comentário da Issue sem apresentar dificuldades, verificando que a exclusão foi refletida na interface.	Link: CT30
CT31	SD	14,0s	O participante fechou a Issue e identificou corretamente a alteração do estado para Closed, sem apresentar dificuldades.	Link: CT31
CT32	SD	10,0s	O participante reabriu a Issue e identificou corretamente o retorno do estado para Open, sem apresentar dificuldades.	Link: CT32
CT33	SD	27,0s	O participante informou um nome de repositório já utilizado e observou a validação apresentada pelo GitHub sem apresentar dificuldades.	Link: CT33
CT34	SD	8,0s	O participante localizou a opção Settings por meio do menu da conta sem apresentar dificuldades.	Link: CT34

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT35	SD	23,0s	O participante utilizou o filtro da página de branches e localizou a branch teste-usabilidade sem apresentar dificuldades.	Link: CT35
CT36	SD	18,0s	Após acessar uma página inexistente, o participante conseguiu retornar ao repositório de teste utilizando os recursos de navegação disponíveis no GitHub.	Link: CT36
CT37	NC	2,21s	O participante não conseguiu localizar, pela interface do GitHub, o caminho para acessar a documentação oficial, indicando que o recurso estava em uma posição diferente da esperada.	Link: CT37
CT38	NC	1,47	O participante não conseguiu localizar a janela de atalhos, pois o acesso ao recurso dependia de um comando de teclado que não estava indicado de forma perceptível na interface.	Link: CT38

Caso	Execução (SD/CD/NC)	Tempo	Descrição	Link (YouTube)
CT39	CD	2,26	O participante apresentou dificuldade durante a navegação pela documentação, principalmente devido à quantidade de informações disponíveis e à organização percebida do conteúdo.	Link: CT39
CT40	SD	28,0s	O participante localizou e acessou a página de suporte do GitHub sem apresentar dificuldades.	Link: CT40

4. Análise de usabilidade

4.1. Avaliação com base nas Heurísticas de Nielsen:

Cada heurística será avaliada pelos quatro CTs associados, considerando a dificuldade observada e a resposta apresentada pelo GitHub Web.

1. Visibilidade do status do sistema — Critério: verificar se o sistema informa claramente seu estado e as mudanças após ações.

Os casos relacionados a essa heurística foram concluídos sem dificuldade. O GitHub apresentou retorno claro após ações como criação de branch, commit, criação de Issue e alteração de seu estado, permitindo ao participante compreender o resultado das ações realizadas.

2. Correspondência entre o sistema e o mundo real — Critério: verificar se termos, símbolos e organização são compreensíveis para o usuário.

A maior parte dos testes apresentou resultados positivos, pois o participante compreendeu conceitos como commits, Issues e Pull Requests. A principal dificuldade ocorreu no CT06, em que não foi simples identificar onde indicar que uma Issue representava um bug.

3. Controle e liberdade para o usuário — Critério: verificar se o usuário consegue cancelar, voltar, editar ou reverter ações.

O participante conseguiu cancelar ou reverter ações em diferentes situações, como edição de arquivos, comentários e estado de Issues. Entretanto, no CT20 não conseguiu localizar a opção necessária para editar o título da Pull Request, impedindo a conclusão da tarefa.

4. Consistência e padrões — Critério: verificar se elementos semelhantes mantêm nomes e comportamentos consistentes.

Os padrões de navegação e os controles utilizados pelo GitHub foram compreendidos na maioria dos casos. Apenas no CT04 foi observada uma pequena dificuldade para identificar o recurso necessário para retornar à página principal do repositório.

5. Prevenção de erros — Critério: verificar se a interface evita ou alerta sobre ações inválidas ou perigosas.

O sistema apresentou mecanismos capazes de impedir ações inválidas ou incompletas, como a criação de uma Issue sem título. No caso da branch inválida, o GitHub impediu a ação, porém não explicou claramente o motivo da restrição.

6. Reconhecimento em vez de memorização — Critério: verificar se opções e pistas visuais reduzem a necessidade de memorização.

Na maior parte dos casos, as opções necessárias estavam visíveis e puderam ser reconhecidas pelo participante. A principal dificuldade ocorreu na localização do recurso para atribuir um responsável à Issue.

7. Flexibilidade e eficiência de uso — Critério: verificar se atalhos, filtros e caminhos rápidos facilitam o uso.

Os testes foram concluídos sem dificuldade. Recursos como Go to file, filtros e atalhos disponíveis na interface facilitaram a localização de arquivos, branches e Issues, tornando a execução das tarefas mais eficiente.

8. Design estético e minimalista — Critério: verificar se a interface destaca informações importantes sem excesso de elementos.

As informações principais foram identificadas facilmente na maioria das páginas. No entanto, no CT18, a quantidade de informações exibidas na Pull Request dificultou a identificação do status de possibilidade de merge.

9. Reconhecimento, diagnóstico e recuperação de erros — Critério: verificar se as mensagens explicam o erro e orientam a correção. Resultado:

O participante conseguiu se recuperar normalmente de situações como filtros sem resultados e páginas inexistentes. A principal dificuldade ocorreu na criação da branch inválida, pois a mensagem apresentada não deixou claro o motivo do erro.

10. Ajuda e documentação — Critério: verificar se recursos de ajuda são fáceis de localizar e úteis.

Esta foi a heurística que apresentou mais dificuldades. O participante não conseguiu concluir os CT37 e CT38 e apresentou dificuldade no CT39, principalmente pela localização pouco intuitiva dos recursos e pela quantidade de informações presentes na documentação.

4.2. Identificação de problemas críticos:

Problema identificado	CT	Heurística	Impacto
Dificuldade para localizar o recurso utilizado para classificar uma Issue como bug.	CT06	H2 – Correspondência entre o sistema e o mundo real	Médio
O GitHub impediu a ação, mas não apresentou de forma clara	CT11 / CT12	H5 / H9	Médio

o motivo da restrição aplicada ao nome da branch.			
Excesso de informações dificultou a identificação do status de possibilidade de merge da Pull Request.	CT18	H8 – Design estético e minimalista	Médio
O participante não conseguiu localizar o recurso utilizado para editar o título da Pull Request.	CT20	H3 – Controle e liberdade para o usuário	Alto
Dificuldade para localizar o recurso utilizado para atribuir um responsável à Issue.	CT26	H6 – Reconhecimento em vez de memorização	Médio
O caminho para acessar a documentação oficial não foi localizado pela interface.	CT37	H10 – Ajuda e documentação	Alto
O participante não encontrou o acesso à janela de atalhos disponível no GitHub.	CT38	H10 – Ajuda e documentação	Médio
A quantidade e a organização das informações dificultaram a navegação pela documentação.	CT39	H10 – Ajuda e documentação	Médio

5. Conclusão e Recomendações

5.1. Conclusão geral:

Os testes realizados no GitHub Web mostraram que o sistema apresenta boa usabilidade na maior parte das tarefas avaliadas. A maioria dos casos foram concluídos sem dificuldade, principalmente em ações relacionadas à navegação, criação de elementos, filtros e alteração de estados.

As principais dificuldades ocorreram na localização de alguns recursos, na clareza de certas validações e no acesso aos recursos de ajuda e documentação. De forma geral, o GitHub atende bem às tarefas mais comuns, mas alguns elementos poderiam ser apresentados de maneira mais clara e acessível.

5.2. Recomendações de melhoria:

Com base nos problemas encontrados, recomenda-se melhorar a visibilidade de recursos como a edição de Pull Requests, atribuição de responsáveis em Issues e informações relacionadas ao status de merge.

Também seria importante tornar as mensagens de validação mais claras, explicando melhor o motivo de uma ação não poder ser concluída. Por fim, os recursos de ajuda, documentação e atalhos poderiam estar mais visíveis e organizados, facilitando sua localização e reduzindo a necessidade de conhecimento prévio do usuário.

6. Checklist do Processo de Teste

Etapa	Pergunta de verificação	Sim	Não	N/A
Planejamento	O sistema e o repositório usados no teste estão definidos?	X		
Planejamento	O objetivo do teste está claro e relacionado à usabilidade do GitHub Web?	X		
Planejamento	A técnica de caixa preta está definida e adequada ao objetivo do teste?	X		
Planejamento	As 10 heurísticas de Nielsen estão contempladas no planejamento?	X		
Planejamento	Há pelo menos 4 casos de teste para cada heurística?	X		
Planejamento	A heurística associada a cada caso de teste é coerente com a ação avaliada?	X		
Planejamento	Os casos de teste avaliam o GitHub Web sem depender de conhecimento sobre código ou estrutura do projeto?	X		
Planejamento	As ações propostas podem ser avaliadas visualmente pela gravação da tela?	X		
Planejamento	Os casos estão organizados em uma sequência clara e executável?	X		
Planejamento	O participante foi definido e atende ao perfil de experiência previsto?	X		
Planejamento	Os papéis da equipe foram definidos?	X		
Preparação	A conta e o repositório exclusivos para os testes estão acessíveis?	X		
Preparação	Os arquivos e conteúdos necessários para os CTs estão disponíveis?	X		
Preparação	As branches necessárias estão no estado esperado antes do início da sessão?	X		
Preparação	Issues e labels necessárias aos CTs estão disponíveis?	X		
Preparação	O estado inicial das Pull Requests está adequado aos casos planejados?	X		
Preparação	A conta de teste possui permissão para	X		

Etapa	Pergunta de verificação	Sim	Não	N/A
	executar todas as ações previstas?			
Preparação	O Firefox está disponível e funcionando para a sessão?	X		
Preparação	O OBS Studio está configurado para gravar a tela durante toda a execução?	X		
Preparação	A forma de obter o tempo de cada CT a partir do vídeo está definida?	X		
Preparação	A tabela de execução e o registro dos links dos vídeos estão preparados?	X		
Execução	O aplicador apresentou somente a tarefa, sem indicar onde clicar?	X		
Execução	A gravação permaneceu ativa durante em cada CT?	X		
Execução	O resultado de cada CT foi registrado na tabela?	X		
Execução	As dificuldades observáveis no vídeo foram registradas quando ocorreram?	X		
Execução	O tempo de cada CT foi obtido a partir da gravação?	X		
Execução	O link do vídeo correspondente foi registrado para cada CT?	X		
Execução	Todos os CTs foram executados ou tiveram uma justificativa registrada?	X		
Análise	Os resultados foram analisados por caso de teste e por heurística?	X		
Análise	Foi verificado se o GitHub apresentou o comportamento e o retorno esperado em cada tarefa?	X		
Análise	Os problemas identificados foram classificados de acordo com seu impacto na usabilidade?	X		
Revisão	O checklist foi reaplicado após a execução dos testes?	X		
Revisão	Os registros, vídeos e resultados foram revisados por outro integrante?	X		

Etapa	Pergunta de verificação	Sim	Não	N/A
Revisão	O relatório final está coerente com as evidências e com o checklist revisado?	X		

7. Referências

- **Nielsen Norman Group – 10 Usability Heuristics**
<https://www.nngroup.com/articles/ten-usability-heuristics/>
- **Nielsen Norman Group – Usability Testing 101**
<https://www.nngroup.com/articles/usability-testing-101/>
- **GitHub Docs – Creating an issue**
<https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>
- **GitHub Docs – Creating a pull request**
<https://docs.github.com/en/pull-requests/how-tos/create-pull-requests/creating-a-pull-request>
- **GitHub Docs – Managing branches**
<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-branches-in-your-repository>
- **GitHub Docs – Keyboard shortcuts**
<https://docs.github.com/pt/get-started/accessibility/keyboard-shortcuts>